Studio 21 Integration API
v1.0.8

Overview

The seamless wallet integration consists of two REST API's: **Games API** and **Wallet API**.

- **Games API** is an API that returns the game URL and the list of available games. Implemented by Studio 21.
- **Wallet API** is an API that triggers to manipulate the user's balance. Implemented by the Operator.

Request signing

All **Games API** requests have to be signed by Operator. All **Wallet API** requests have to be signed by Studio 21.

Integration requirements:

- The Operator generates a private/public key pair (**RSA 1024 bit**) and sends the public key to Studio 21.
- Studio 21sends its public key (**RSA 1024 bit**) to the Operator.
- The body of all requests will be signed with **RSA-SHA256** using the respective private key and encoded to **Base64**.
- The signature will be placed in the **Casino-Signature** header.
- The Operator needs to verify all **Wallet API** requests using the public key provided by Studio 21. Studio 21 verifies all **Games API** requests using the public key provided by the Operator.

Example:

- Private key

```
-----BEGIN PRIVATE KEY-----
MIICdwIBADANBgkqhkiG9w0BAQEFAASCAmEwggJdAgEAAoGBAMnKR1hXP8ohB7Gc
83sOIqiBtv+hA2JlcaxTMv8+nzTbkqJTPWwzHQHkw9Zuo63evxXCqU+Bh0VMWAEX
ks+yoaS80kJJ4u7BcPzwbSy9rEC1KsycJbsbgEFdWpgFWzQZResaNkV3iMc9wBBm
I2GHfXahcHRz9AUA6oCVZDoGA57vAgMBAAECgYEAjscMUipx3TAfxIF8Gc63oSke
9aLUxTNoCflYdRe3CGxwRWDGbh+AX82g78GbGLQp2ljA+Zt5vgO1pRgr7b/D9Iv8
dTu4VCfBW9aYDQ1kldSyCQ710IdN1GcYsQIgMaRrvjlVjjqvaAQQaee4UEEagHYC
lRAr+lQQbu55rsTfjPECQQD54vvAa7lVJJqsB3WItzNHN2NXpAMzfVjKX5L5pnKM
7mnEB3906+rL+EC+Op+Ca7TkDUzGxIIuxgcbfvsiygBnAkEAzroSSFtm0lyxuKLX
u3n2PbTL/+sYOw03nUJyIrcXs2M2B0mn+IXefAkcLW5z30Jt+4iUUGTBXVNx0F5D
bZU4OQJBALeaYHGEdRsrFv5ZSLGYMfQcrUBiwPtfGRMw4BPQiZAlSM8A11R4Vv9h
O2jKOBMiwjlh0EynbCOj2usIBMwN8fcCQFRbW5CJM7kNuDvmv2+yQghGGaSx025t
Y09ZyOdfnV1HU8m/hbqFLhehRA1J8CCfo++reRIhO0KRBXWIsa0q4gkCQHnf5QSf
amts2vcLXImSIPQ/s8HmljCoKlv8g5O4/Or2qojtKAw8psk0gUYizi4+9gBQ9T5j
DvwVR+Alc3Nx/VY=
-----END PRIVATE KEY-----
```

- Request

```
{"user":"User12345","token":"afb889a5-176b-4a6d-bf5b-
ed4f8a3f7585","platform":"GPL_DESKTOP","meta":{"rating":10,"oddsType":"decimal"},"lobby_url":"https://
example.com/lobby","lang":"en","ip":"142.245.172.168","game_id":777,"game_code":"mega_crash","deposit_
url":"https://example.com/deposit"}
```

- Correct **Casino-Signature**

```
FCXxiVpY2crAXG5GPOYQOkepAJdopAkprOZ0BHmhHtukHyZLdMOrkGAROBp5SKWUTeZKzez9fWCVlsKz4Ww55JIG2GvjqkL6aL4oeS
A3qprt+Sog8g3c3NuEJPZfqS/xNtQLWYwrXCo0gErYMFqOGN6CrMoyhia/il3hdGlfrU4=
```

Request consistency

Wallet API requests must be idempotent and include a transaction_uuid field. The Operator must ensure that requests with the same transaction_uuid are not processed twice while the response must be the same for all duplicate requests. The Operator may ignore the idempotency requirement for **/wallet/balance** call.

Gameplay

Once you meet the prerequisites, the process is as follows:

1. Obtain a game URL from Studio 21.
2. Direct the customer to the URL provided by Studio 21.
3. Respond to in-game events sent by Studio 21 and update the customer's balance.

Below is an example of interaction flow between the customer, operator, Studio 21, and game provider in REAL mode:

1. The Operator generates and stores a unique game session token. The token needs to be per session based with Casino. It is not per session at your platform, thus new token is required when a user revisit casino during same session at your platform. The session token acceptable expire time is 30 mins.
2. The Operator makes a Game API call **/games/url** and passes the generated token along with the other request parameters. **NOTE**: Operator needs to be ready to respond to balance API calls even before getting the game URL back.
3. When the game URL is returned, the Operator uses it to direct the player to the game (for example, launch it in iframe or redirect player to the URL).
4. When the game URL is loaded in the browser, Studio 21 server makes a Wallet API call **/wallet/balance** to the Operator's server.
5. The Operator verifies the token against the stored token and returns the user's balance. The user can then place a bet.
6. When the user attempts to place a bet, a Wallet API call **/wallet/bet** is triggered on the Operator's server.
7. The Operator verifies the token, ensures that the user has enough money for this bet, decreases the user's balance by the bet amount, and returns the updated user's balance.
8. If the user wins, the Wallet API triggers a call **/wallet/win** on the Operator's server.
9. The Operator verifies the token, increases the user's balance by the win amount, and returns the updated user's balance.
10. Behavior in case of user's loss depends on game provider's internal logic. Possible options:
    - Nothing is sent.
    - Call **/wallet/win** with zero amount.

Response Statuses

Status Code	Description
RS_OK	Successful response.
RS_ERROR_UNKNOWN	General error status, for cases without a special error code.
RS_ERROR_INVALID_SIGNATURE	The signature does not match the public key.
RS_ERROR_INVALID_PARTNER	Partner is disabled or incorrect partner_id is sent.
RS_ERROR_INVALID_TOKEN	Token unknown to Operator's system. Please, note that there is a different status for expired tokens.
RS_ERROR_INVALID_GAME	Unknown game_code. NOTE: in case of game providers with game lobby (Live Dealers), user can switch games within one game session. We track such changes and send game_code of the game which the user is actually playing at the moment. Note that game_code may change within one game session.
RS_ERROR_WRONG_CURRENCY	Transaction currency differs from User's wallet currency.
RS_ERROR_NOT_ENOUGH_MONEY	Not enough money on User's balance to place a bet. Please send the actual balance together with this status.
RS_ERROR_USER_DISABLED	User is disabled/locked and can't place bets.
RS_ERROR_TOKEN_EXPIRED	Session with specified token has already expired. NOTE: token validity MUST NOT be validated in case of wins and rollbacks, since they might come long after the bets.
RS_ERROR_WRONG_SYNTAX	Received request doesn't match expected request form and syntax.
RS_ERROR_WRONG_TYPES	Type of parameters in request doesn't match expected types.
RS_ERROR_DUPLICATE_TRANSACTION	A transaction with same transaction_uuid was sent, but there was a different amount, currency, round, user or game.
RS_ERROR_TRANSACTION_DOES_NOT_EXIST	Returned when the bet referenced in win request can't be found on Operator's side (wasn't processed or was rolled back). If you received rollback request and can't find the

	transaction to roll back, respond RS_OK

Games API

Studio 21 offers several ways for the Operator to display games on their website. These endpoints are configured on Studio 21's side.

POST /games/url

1. Returns the Landing URL of the chosen game. Operator has to forward User to returned URL.
2. There are several ways to forward the User:
    a. Embed URL into iframe on your site;
    b. Redirect User to URL;
    c. Open URL in new window/tab of browser.
3. Game code is interchangeable with game_id. Either game_id or game_code must be present in request

Name	Type	Description
Header		
Casino-Signature*	String	**RSA-SHA256** is used to sign the request body using the Operator's private key. The signature is validated on Studio 21 side using the public key associated with the provided partner_id.
Content-Type*	Application/json	The body's datatype is a JSON object.
Body		
user*	String	Unique userId in Operator's system.
nickname	String	If the user* is a UUID you can send the user's login name or some sort of a short name to be associated with the user and displayed in the games' interface.
token*	String	Operator's back end generates a token associated with the User, game, his or her current currency and maybe other parameters depending on the Operator's preferences. The token acts as an ID parameter for the game.
partner_id*	String	Unique partner id provided by Studio 21.
platform*	String	Type of device (GPL_Desktop or GPL_Mobile) as it affects game layout.
lobby_url*	String	Most games have a Home button that redirects the User back to the lobby. Put the lobby URL here. Example: https://example.com/lobby
lang*	String	ISO 639-1 language code. Example: en
ip*	String	User's IP address. Example: 23.125.72.138
game_id	Long	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_code. Either game_id or game_code must be present in the request.
game_code	String	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_id. Either game_id or game_code must be present in the request.
meta	Object	Additional parameters, can be used for certain game suppliers. It depends on Operator's choice of Game Providers.
deposit_url	String	Some games have button that redirects the User to a page where he can deposit money. Put your deposit page URL here. Example: https://example.com/deposit
country*	String	User's country ISO 3166-1 code. Example: UK
currency*	String	ISO 4217 currency code. It contains the currency enabled for the operator. Example: USD

Request example:

body* Example Value:
object {
(body) "user": "User12345",
"token": "afb889a5-176b-4a6d-bf5b-ed4f8a3f7585",
"platform": "GPL_DESKTOP",
"meta": {
"rating": 10,
"oddsType": "decimal"
},
"lobby_url": "https://example.com/lobby",
"lang": "en",
"ip": "142.245.172.168",
"game_id": 777,

```
        "game_code": "mega_crash",
        "deposit_url": "https://example.com/deposit",
        "country": "US",
        "currency": "USD"
}
```


Response example:

Code 200 Description OK

Example Value:


```
{
        "url": "https://example.com/game?session=ddbab63b-3967-4807-93e2-2b0ffdec0a3c"
}
```


Fields:

```
{
        url String Landing URL of the chosen game.
}
```

POST/games/list

This endpoint lists of games available for the Operator. The list should be downloaded and saved in the local system. The Operator should configure games according to available thumbnails on its site. This list should not be called frequently or on a per-user basis.

Name	Type	Description
Header		
Casino-Signature*	String	**RSA-SHA256** is used to sign the request body using the Operator's private key. The signature is validated on Studio 21 side using the public key associated with the provided partner_id.
Content-Type*	Application/json	The body's datatype is a JSON object.
Body		
partner_id*	String	Unique partner id provided by Studio 21.
Response Body		

Request example:

body* object (body) Example Value:


```
{
        "partner_id": "MegaCasino777"
}
```


Fields:

```
{
        partner_id* Unique identifier of Operator in dream casino system. Example: MegaCasino777
}
```

Response example:

Code 200 Description OK

Example Value:


```
[
        {
                "url_thumb": "https://picture-hosting.com/blackjack/thumb.png",
                "url_background": "https://picture-hosting.com/blackjack/bg.png",
                "product": "OneTouch",
                "platforms": [
                        "GPL_DESKTOP",
                        "GPL_MOBILE"
                ],
                "name": "Blackjack Classic",
                "game_id": 615,
                "game_code": "ont_blackjackclassic",
                "freebet_support": true,
                "enabled": true,
                "category": "Blackjack",
                "blocked_countries": [
                        "AU",
                        "US",
                        "UK"
                ],
                "release_date": "2017-11-05",
                "scatters": false,
```

```
                "in_game_freebets": false,
                "volatility": 1,
                "rtp": "98.8",
                "paylines": 0,
                "hit_ratio": "45.50",
                "certifications": [
                        "CURACAO",
                        "MGA"
                ],
                "languages": [
                        "eng",
                        "jpn",
                        "kor",
                        "deu"
                ],
                "theme": [
                        "Vegas",
                        "Blue",
                        "Wood"
                ],
                "technology": [
                        "HTML5"
                ]
        }
]
```


Wallet API

The Operator is expected to provide implementation for this API defined by Studio 21. The API should process the following requests from Studio 21. These endpoints are configured on Operator side.

POST/wallet/balance

This end-point is called when the player's balance is needed. The Operator have to return the player's current balance. The game ID is provided to help the Operator with player activity statistics.

Name	Type	Description
Header		
Casino-Signature*	String	**RSA-SHA256** is used to sign the request body using Studio 21's private key. The signature is validated on the Operator's side using Studio 21's public key.
Content-Type*	Application/json	The body's datatype is a JSON object.
Request Body		
user*	String	Unique user ID in the Operator's system.
token*	String	Operator's back end generates a token associated with the User, game, his or her current currency and maybe other parameters depending on the Operator's preferences. The token acts as an ID parameter for the game.
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
game_id	Long	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_code. Either game_id or game_code must be present in the request.
game_code	String	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_id. Either game_id or game_code must be present in the request.
Response Body		
user*	String	Unique user ID in the Operator's system.
status*	String	Response Status. Example: RS_OK
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e

balance*	Long	We use integers to represent the amount of money. To convert floating point value to integer - multiply it by 100000. Example: $3.56 must be represented as 356000
currency*	String	ISO 4217 currency code. It contains the currency enabled for the operator. Example: USD

Response example:

Code Description
200 OK

Example Value:


```
{
    "user": "User12345",
    "status": "RS_OK",
    "request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
    "balance": 356000,
    "currency": "USD"
}
```


POST/wallet/bet

This end-point is called when the User places a bet (debit).

- Operator is expected to decrease player's balance by the provided amount and return the new balance.
- Each bet has transaction_uuid which is unique identifier of this transaction. Before altering the User's balance, the Operator must verify that the bet has not been processed before. There might be Retry Policy: In case of network fail (HTTP 502, timeout, nxdomain, etc.), we will retry 3 times with 1 sec of timeout. If we do not receive 200 HTTP status, this transaction will be counted as failed and rollback will be generated (to ensure that failed bet hadn't affected User's balance). This rollback will be retried 500 times (or less if we get logical response) with exponential back off.

Name	Type	Description
Header		
Casino-Signature*	String	**RSA-SHA256** is used to sign the request body using Studio 21's private key. The signature is validated on the Operator's side using Studio 21's public key.
Content-Type*	Application/json	The body's datatype is a JSON object.
Request Body		
user*	String	Unique user ID in the Operator's system.
transaction_uuid*	String	Unique transaction identifier. An id of a business logic action (transaction) that needs to be stored on both sides for at least 4 months (for reconciliation purposes). The Operator has to respond idempotently on each transaction_uuid. Action with same transaction_uuid shouldn't be processed more than once.
token*	String	Operator's back end generates a token associated with the User, game, his or her current currency and maybe other parameters depending on the Operator's preferences. The token acts as an ID parameter for the game.
supplier_user*	String	The name of the user in the Provider's system (in case Operator needs to find user in the Provider's back office or report problem with the user). If value is NULL, the Operator can search for their own user_id.
round_closed*	Boolean	True /False. Denotes when the round is closed.
round_id*	String	Used to relate all bets and wins in one round. All transactions related to the same round will have the same value in this field. It's not unique through whole system. Uniqueness depends on provider's RGS logic, however, game + user + round yields very high uniqueness.
reward_uuid	String	Standard 16-byte UUID. Unique reward id in Studio 21 side.
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
is_free*	Boolean	True/False. Flag which shows that transaction was generated by a promotional tool e.g., free spins, etc.
is_aggregated	Boolean	True/False. Flag which shows that related freebet transactions identified with the field reward_uuid has been aggregated into a single transaction. E.g player was rewarded ten (10) freespins, but we send you a single transaction request with field is_aggregated set to true.
game_id	Long	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_code. Either game_id or game_code must be present in the request.
game_code	String	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint.

		Interchangeable with game_id. Either game_id or game_code must be present in the request.
currency*	String	ISO 4217 currency code. It contains the currency enabled for the operator. Example: USD
meta*	Object	Placeholder for metadata related to transaction, such as type of bet, value, time, etc. Differs from game to game. Not relevant for transaction processing procedure but could be useful for statistics or activity backtracking.
amount*	Long	We use integers to represent the amount of money. To convert floating point value to integer - multiply it by 100000. Example: $3.56 must be represented as 356000
Response Body		
user*	String	Unique user ID in the Operator's system.
status*	String	Response Status. Example: RS_OK
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
balance	Long	We use integers to represent the amount of money. To convert floating point value to integer - multiply it by 100000. Example: $3.56 must be represented as 356000
currency	String	ISO 4217 currency code. Example: USD

Request example:

body* object (body)

Example Value:


```
{
    "user": "User12345",
    "transaction_uuid": "d7e33f5e-c6e8-4df5-b190-3c032fba071a",
    "token": "afb889a5-176b-4a6d-bf5b-ed4f8a3f7585",
    "supplier_user": "mc_54321",
    "round_closed": true,
    "round_id": "rEM967985Z6eF",
    "reward_uuid": "a28f93f2-98c5-41f7-8fbb-967985acf8fe",
    "request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
    "is_free": false,
    "is_aggregated": false,
    "game_id": 777,
    "game_code": "mega_crash",
    "currency": "USD",
    "meta": {
        "multiplier": "1.3"
    },
    "amount": 100000
}
```


Response example:

Code	Description
200	OK

Example Value:


```
{
    "user": "User12345",
    "status": "RS_OK",
    "request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
    "balance": 356000,
    "currency": "USD"
}
```


POST/wallet/win

This end-point is called when the User wins (credit).

- The Operator is expected to increase player's balance by the provided amount and return a new balance. reference_transaction_uuid shows the related bet.
- Each win has transaction_uuid which is unique identifier of this transaction. Before altering the User's balance, the Operator must verify that the bet has not been processed before.
- Retry Policy: In case of network fail (HTTP 502, timeout, nxdomain, etc.) we will retry 3 times with 1 sec of timeout. The rest of retry logic is left to provider's RGS: the retries may continue indefinitely or the bet may be rolled back, and the money returned back to user.

Name	Type	Description
Header		
Casino-Signature*	String	**RSA-SHA256** is used to sign the request body using Studio 21's private key. The signature is validated on the Operator's side using Studio 21's public key.
Content-Type*	Application/json	The body's datatype is a JSON object.
Request Body		
user*	String	Unique user ID in the Operator's system.
transaction_uuid*	String	Unique transaction identifier. An id of a business logic action (transaction) that needs to be stored on both sides for at least 4 months (for reconciliation purposes). The Operator has to respond idempotently on each ransaction_uuid. Action with same transaction_uuid shouldn't be processed more than once.
token*	String	Operator's back end generates a token associated with the User, game, his or her current currency and maybe other parameters depending on the Operator's preferences. The token acts as an ID parameter for the game.
supplier_user*	String	The name of the user in the Provider's system (in case Operator needs to find user in the Provider's back office or report problem with the user). If value is NULL, the Operator can search for their own user_id.
round_closed*	Boolean	True /False. Denotes when the round is closed.
round_id	String	Used to relate all bets and wins in one round. All transactions related to the same round will have the same value in this field. It's not unique through whole system. Uniqueness depends on provider's RGS logic, however, game + user + round yields very high uniqueness.
reward_uuid	String	Standard 16-byte UUID. Unique reward id in Studio 21 side.
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
reference_transaction_uuid*	String	Unique identifier of the transaction that this transaction is referencing. In case of rollback, this field will contain transaction_uuid of the transaction that needs to be rolled back. In case of win, there will be transaction_uuid of the bet related to this win.
is_free*	Boolean	True/False. Flag which shows that transaction was generated by a promotional tool e.g., free spins, etc.
is_aggregated	Boolean	True/False. Flag which shows that related freebet transactions identified with the field reward_uuid has been aggregated into a single transaction. E.g player was rewarded ten (10) freespins, but we send you a single transaction request with field is_aggregated set to true.
game_id	Long	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_code. Either game_id or game_code must be present in the request.
game_code	String	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_id. Either game_id or game_code must be present in the request.
currency*	String	ISO 4217 currency code. It contains the currency enabled for the operator. Example: USD
meta*	Object	Placeholder for metadata related to transaction, such as type of bet, value, time, etc. Differs from game to game. Not relevant for transaction processing procedure but could be useful for statistics or activity backtracking.
amount*	Long	We use integers to represent the amount of money. To convert floating point value to integer - multiply it by 100000. Example: $3.56 must be represented as 356000
Response Body		
user*	String	Unique user ID in the Operator's system.
status*	String	Response Status. Example: RS_OK
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
balance	Long	We use integers to represent the amount of money. To convert floating point value to integer - multiply it by 100000. Example: $3.56 must be represented as 356000
currency	String	ISO 4217 currency code. Example: USD

Request example:

body* Example Value:
object {
(body) "user": "User12345",
"transaction_uuid": "457d30f2-35cc-44d8-b1df-47fe44cff506",
"token": "afb889a5-176b-4a6d-bf5b-ed4f8a3f7585",
"supplier_user": "mc_54321",
"round_closed": true,
"round_id": "rEM967985Z6eF",
"reward_uuid": "a28f93f2-98c5-41f7-8fbb-967985acf8fe",
"request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
"reference_transaction_uuid": "d7e33f5e-c6e8-4df5-b190-3c032fba071a",
"is_free": false,
"is_aggregated": false,
"game_id": 777,
"game_code": "mega_crash",
"currency": "USD",
"bet": "zero",
"amount": 130000
}

Response example:

Code Description
200 OK

Example Value:
{
"user": " User12345",
"status": "RS_OK",
"request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
"balance": 486000,
"currency": "USD"
}

POST/wallet/rollback

This end-point is called when there is a need to roll back the referenced transaction. The Operator is expected to find the referenced transaction, roll back its effect and return the player's new balance.

Name	Type	Description
Header		
Casino-Signature*	String	**RSA-SHA256** is used to sign the request body using Studio 21's private key. The signature is validated on the Operator's side using Studio 21's public key.
Content-Type*	Application/json	The body's datatype is a JSON object.
Request Body		
user*	String	Unique user ID in the Operator's system.
transaction_uuid*	String	Unique transaction identifier. An id of a business logic action (transaction) that needs to be stored on both sides for at least 4 months (for reconciliation purposes). The Operator has to respond idempotently on each ransaction_uuid. Action with same transaction_uuid shouldn't be processed more than once.
token*	String	Operator's back end generates a token associated with the User, game, his or her current currency and maybe other parameters depending on the Operator's preferences. The token acts as an ID parameter for the game.
round_closed*	Boolean	True /False. Denotes when the round is closed.
round_id	String	Used to relate all bets and wins in one round. All transactions related to the same round will have the same value in this field. It's not unique through whole system. Uniqueness depends on provider's RGS logic, however, game + user + round yields very high uniqueness.
reward_uuid	String	Standard 16-byte UUID. Unique reward id in Studio 21 side.
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
reference_transaction_uuid*	String	Unique identifier of the transaction that this transaction is referencing. In case of rollback, this field will contain transaction_uuid of the transaction that needs to be rolled back. In case of win, there will be transaction_uuid of the bet related to this win.
game_id	Long	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_code. Either game_id or game_code must be present in the request.
game_code	String	Unique game ID in Studio 21 system. Can be obtained from **/game/list** endpoint. Interchangeable with game_id. Either game_id or game_code must be present in the request.

Response Body		
user*	String	Unique user ID in the Operator's system.
status*	String	Response Status. Example: RS_OK
request_uuid*	String	Standard 16-byte UUID. This ID can be seen as network layer action. An ID of an action that is generated for each call to the Operator. Used to sync Studio 21 and the Operator's sides for debugging purposes. The Operator has to respond with the same request_uuid as the one received with the request. Example: f0a4c6e7-7d9b-4e1d-9f2c-5f8b5a6c8d1e
balance	Long	We use integers to represent the amount of money. To convert floating point value to integer - multiply it by 100000. Example: $3.56 must be represented as 356000
currency	String	ISO 4217 currency code. Example: USD

Request example:

body* object (body) Example Value:


```
{
    "user": "User12345",
    "transaction_uuid": "457d30f2-35cc-44d8-b1df-47fe44cff506",
    "token": "afb889a5-176b-4a6d-bf5b-ed4f8a3f7585",
    "round_closed": true,
    "round_id": "rEM967985Z6eF",
    "request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
    "reference_transaction_uuid": "d7e33f5e-c6e8-4df5-b190-3c032fba071a",
    "game_id": 777,
    "game_code": "mega_crash"
}
```


Response example:

Code Description

200 OK

Example Value:


```
{
    "user": "User12345",
    "status": "RS_OK",
    "request_uuid": "c8d1e5f8-b5a6-4e1d-9f2c-7d9bf0a4c6e7",
    "balance": 456000,
    "currency": "USD"
}
```


NOTE:

1. Fields marked with * are required.
2. All requests and responses have application/json content type.
3. If Wallet API response status is other than **RS_OK** then the response must contain **user**, **status** and **request_uuid** fields.

Appendix A

RSA key pair generation

Generate RSA private key

```
openssl genrsa -out {path_to_private_pem_file} 1024
```

Example:

```
openssl genrsa -out private-key.pem 1024
```

Generate RSA public key

```
openssl rsa -pubout -in {path_to_private_pem_file} -out {path_to_public_pem_file}
```

Example:

```
openssl rsa -pubout -in private-key.pem -out public-key.pem
```

Appendix B

Example code

Sign with Java

```
import java.nio.charset.StandardCharsets;
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.Signature;
import java.security.spec.PKCS8EncodedKeySpec;
import java.util.Base64;

import org.slf4j.Logger;

public String makeSignature(String privateKeyStr, String requestStr) {
  String result = null;

  try {
    final byte[] privateKeyEncoded = Base64.getDecoder().decode(privateKeyStr);
    KeyFactory keyFactory = KeyFactory.getInstance("RSA");
    PKCS8EncodedKeySpec privateKeySpec = new PKCS8EncodedKeySpec(privateKeyEncoded);
    final PrivateKey privateKey = keyFactory.generatePrivate(privateKeySpec);

    Signature privateSignature = Signature.getInstance("SHA256withRSA");
    privateSignature.initSign(privateKey);
    privateSignature.update(requestStr.getBytes(StandardCharsets.UTF_8));
    result = Base64.getEncoder().encodeToString(privateSignature.sign());
    log.info("makeSignature() signature(b64): {}", result);
  } catch (Exception e) {
    log.error("makeSignature()", e);
  }

  return result;
}
```

Validate signature with Java

```
import java.io.ByteArrayInputStream;
import java.nio.charset.StandardCharsets;
import java.security.KeyFactory;
import java.security.PublicKey;
import java.security.Signature;
import java.security.spec.X509EncodedKeySpec;
import java.util.Base64;

import org.slf4j.Logger;

public boolean checkSignature(String publicKeyStr, String requestStr, String signatureStr) {
  try {
    final byte[] signature = Base64.getDecoder().decode(signatureStr);
    final byte[] publicKeyEncoded = Base64.getDecoder().decode(publicKeyStr);
    KeyFactory keyFactory = KeyFactory.getInstance("RSA");
    X509EncodedKeySpec publicKeySpec = new X509EncodedKeySpec(publicKeyEncoded);
    final PublicKey publicKey = keyFactory.generatePublic(publicKeySpec);

    Signature publicSignature = Signature.getInstance("SHA256withRSA");
    publicSignature.initVerify(publicKey);
    try (InputStream is = new ByteArrayInputStream(requestStr.getBytes())) {
      byte[] buffer = new byte[128];
      int length;
      while ((length = is.read(buffer)) > 0) {
        publicSignature.update(buffer, 0, length);
      }

      return publicSignature.verify(signature);
    }
  } catch (Exception e) {
    log.error("checkSignature()", e);
```

```
  }
  return false;
}
```

Sign with Node.js

```
const crypto = require('crypto');
const fs = require('fs');

// Load your RSA private key (PEM format)
const privateKey = fs.readFileSync('private-key.pem', 'utf8');

// The data you want to sign
const data = '{"partner_id":" MegaCasino777"}';

// Create signer object
const signer = crypto.createSign('RSA-SHA256');
signer.update(data);
signer.end();

// Generate signature in base64 format
const signature = signer.sign(privateKey, 'base64');

console.log('Casino-Signature: ', signature);
```

Validate signature with Node.js

```
const crypto = require('crypto');
const fs = require('fs');

const publicKey = fs.readFileSync('public-key.pem', 'utf8');

// The data whose signature you want to verify
const data = '{"partner_id":" MegaCasino777"}';

// The signature you want to verify
const signature =
'M2/ydAUYzF5JjieATDN084/cuBSupVyWfSeZLRBvGdPToGMvDJdR9rH/KLUojZZr0nU8n/XZj7kHJIuFwy5kof4H+aSRsNvrth8Zs
BqeiGXBYQDif6bCwPqpnTw+/FIvVmuw8a3kpaiQq3ClJg1kFb1n5l/tS3FAw1/zAZfifLo=';

const verifier = crypto.createVerify('RSA-SHA256');
verifier.update(data);
verifier.end();

const isValid = verifier.verify(publicKey, signature, 'base64');

console.log('Is signature valid:', isValid);
```

Sign with .Net (C#)

```
using System.Security.Cryptography;

static string SignDataToBase64String(byte[] data, RSA privateKey)
{
  return Convert.ToBase64String(SignData(data, privateKey));
}

static byte[] SignData(byte[] data, RSA privateKey)
{
  return privateKey.SignData(data, HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
}
```

Validate signature with .Net (C#)

```
using System.Security.Cryptography;

static bool VerifySignatureFromBase64String(byte[] data, String signature, RSA publicKey)
{
  return VerifySignature(data, Convert.FromBase64String(signature), publicKey);
```

```
}

static bool VerifySignature(byte[] data, byte[] signature, RSA publicKey)
{
  return publicKey.VerifyData(data, signature, HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
}
```

Load public or private key from a PKCS8 PEM file with .Net (C#)

PEM file beginning with -----BEGIN PRIVATE KEY----- / -----BEGIN PUBLIC KEY-----

```
using System.Security.Cryptography;

static RSA LoadKeyFromPem(string path)
{
  string pem = File.ReadAllText(path);
  RSA rsa = RSA.Create();
  rsa.ImportFromPem(pem.ToCharArray());
  return rsa;
}
```

Load private key from a PKCS1 PEM file with .Net (C#)

PEM file beginning with -----BEGIN RSA PRIVATE KEY-----

```
using System.Security.Cryptography;

static RSA LoadPKCS1PrivateKeyFromPem(string path)
{
  string pem = File.ReadAllText(path);
  RSA rsa = RSA.Create();
  rsa.ImportRSAPrivateKey(pem.ToCharArray());
  return rsa;
}
```